LATTICE · FEATURE DECK

Native state machines in plain markdown.

Numbered list, inline-code transitions, palette-blind SVG. No Mermaid, no charting library, no layout engine.

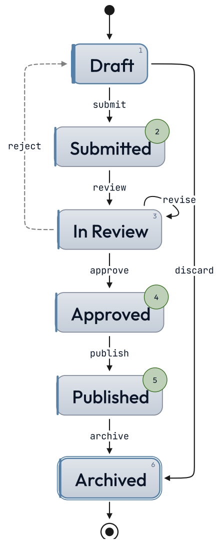
What this deck shows.

A finite-state machine authored as an ordered list. Each top-level item is a state; the index becomes a stable ref. Nested bullets carry the outgoing transitions, each a single inline-code arrow like `submit => 2` or `revise => self`. Whitespace inside the inline code is insignificant. The browser measures the laid-out nodes and draws the SVG edges, so it sizes to any content. Two orthogonal modifier classes: direction — `lr` (left-to-right, Mermaid `direction LR`) or the default top-to-bottom — and presentation — `inline` (transitions as chips, no SVG). They compose. `dark` composes on top.

SUBMISSION LIFECYCLE

Document approval flow.

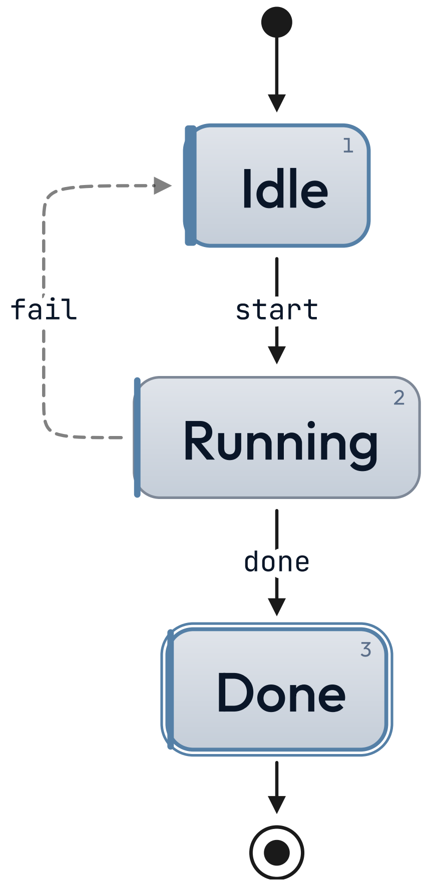
How a draft moves from author to archive.



Rejected drafts return to the author; revisions stay in review.

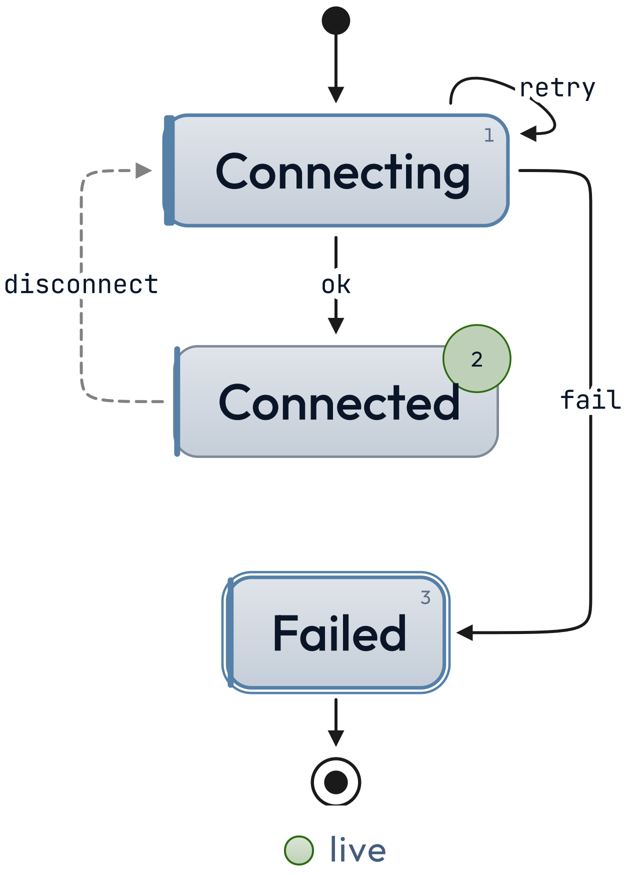
on-track done live

Job runner.

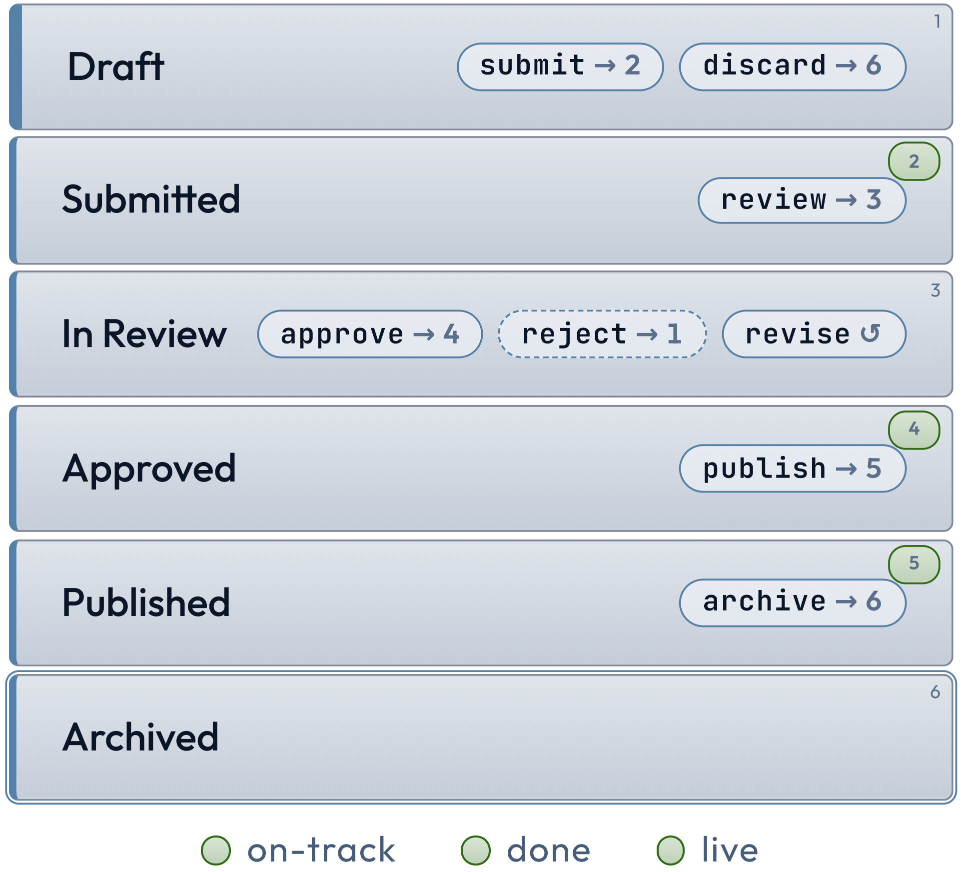


Minimal — three states, linear flow

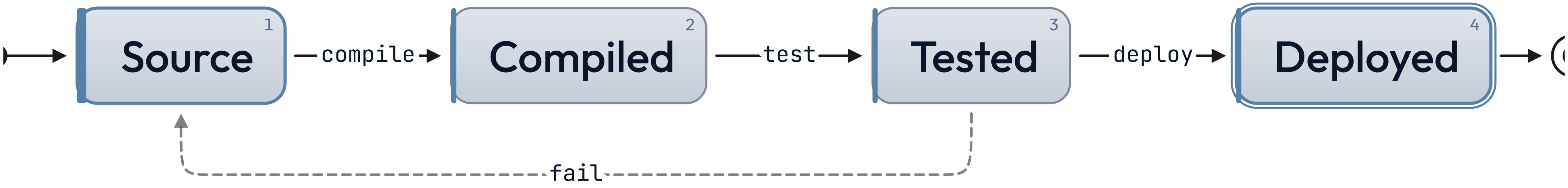
Connection retry.



Inline rendering.



Build pipeline.



lr — left-to-right (Mermaid direction LR)

Why a native state chart.

Mermaid's `stateDiagram-v2` works, but theming it cleanly requires a CSS override cascade with `!important` (see `docs/theming.md`). The native chart uses palette tokens directly — `var(--cat-1-fill)`, `var(--diagram-stroke)`, `var(--diagram-line)`, `var(--cat-on-fill)` — and inherits dark / light, every theme, automatically. No mmdc subprocess, no opaque SVG class names, no version coupling. The trade-off: no hierarchical states, no parallel regions, no guards in v1. When you need those, `<!-- _class: diagram -->` is the Mermaid escape hatch.

LATTICE · STATE CHART

Numbered authoring is the layout.